Hash Functions & MAC

Cybersecurity – UniPR | Riassunto per l'esame

Domande coperte: Q9, Q10, Q11, Q12 — Esercizi: Es.10

1. Cos'è una Hash Function

Una **funzione hash crittografica** H(.) è una funzione che prende in input un messaggio m di lunghezza arbitraria e produce in output un valore di lunghezza fissa chiamato **hash**, **digest** o **fingerprint**.

Proprietà fondamentale: H(m) = h (m → h è facile, h → m è computazionalmente impossibile)

2. Proprietà di una Hash Sicura (Q9)

Proprietà	Definizione	Attacco sconfitto
Preimage resistance (one-way)	Dato h, impossibile trovare m tale che H(m) = h	Brute force su h
Second preimage resistance	Dato m1, impossibile trovare m2 ≠ m1 tale che H(m1) = H(m2)	Attacco su messaggio noto
Collision resistance	Impossibile trovare m1 ≠ m2 tale che H(m1) = H(m2)	Birthday attack

> **Nota:** collision resistance implica second preimage resistance, ma non il contrario. La proprietà più forte da garantire è la collision resistance.

3. Usi delle Hash Functions (Q10)

• **Message Authentication (MAC/HMAC)**: Verificare che il messaggio non sia stato alterato e provenga dal mittente corretto.

• **Digital Signature**: Si firma l'hash del messaggio (non il messaggio intero) per efficienza: Sign(KR, H(m)).

• **Password Storage**: Le password non vengono salvate in chiaro ma come hash: H(password). Spesso con salt per evitare rainbow table attacks.

• **Integrity Check**: Verificare che un file scaricato non sia corrotto o modificato (es. checksum SHA-256).

• **Pseudorandom Number Generation**: Le hash function possono essere usate come base per generatori di numeri pseudocasuali.

• **Key Derivation**: Derivare chiavi crittografiche da una password o da un segreto condiviso.

4. Message Authentication Code (MAC) (Q11)

Un **MAC** è un tag crittografico calcolato su un messaggio usando una **chiave segreta condivisa** KAB. Garantisce contemporaneamente **autenticità** e **integrità** del messaggio.

Schema:

```
MAC = C(KAB, m)
```

Il mittente A invia (m, MAC). Il ricevente B ricalcola il MAC e confronta: se coincide, il messaggio è autentico e integro.

> **Differenza con firma digitale:** il MAC usa una chiave simmetrica condivisa (solo A e B possono verificarlo). La firma digitale usa chiave privata (chiunque con la chiave pubblica può verificarla) e garantisce anche il non-ripudio.

5. HMAC — Schema e Conteggio Passi (Q12)

HMAC (Hash-based MAC) costruisce un MAC a partire da una funzione hash H e una chiave K. Risolve i problemi di sicurezza del semplice H(K || m).

Formula HMAC:

```
HMAC(K, m) = H( (K XOR opad) || H( (K XOR ipad) || m ) )
```

Parametri:

- ipad = 0x36 ripetuto per la lunghezza del blocco
- opad = 0x5C ripetuto per la lunghezza del blocco
- K = chiave (se più lunga del blocco, viene prima hashata)
- m = messaggio di lunghezza N·M (N blocchi di M bit ciascuno)

Conteggio dei passi C (funzione di compressione):

H processa il messaggio in blocchi di dimensione M usando la funzione C. Ogni chiamata a H richiede un certo numero di passi C:

Operazione	Passi C richiesti	Spiegazione
H((K XOR ipad) || m)	N + 1	1 blocco per (K XOR ipad) + N blocchi del messaggio m
H((K XOR opad) || risultato)	2	1 blocco per (K XOR opad) + 1 blocco per il risultato interno
TOTALE	**N + 3**	**Somma dei passi delle due chiamate a H**

Risposta alla Q12: Per un messaggio m di lunghezza N·M, il numero di chiamate alla funzione C è **N + 3**.

6. Esercizio 10 — Schemi con Hash + Chiave Condivisa

i) Schema di autenticazione A → B:

A e B condividono KAB. A vuole dimostrare la propria identità a B:

```
1. B invia a A un nonce (numero casuale) r
```

```
2. A calcola: tag = H(KAB || r)
```

```
3. A invia tag a B
```

```
4. B ricalcola H(KAB || r) e confronta con tag ricevuto
```

```
→ Se coincide: A è autenticato (solo chi conosce KAB può produrre tag)
```

ii) Invio messaggio con autenticazione + integrità:

A vuole inviare m a B garantendo che il messaggio sia autentico e integro:

```
Lato A (mittente):
```

```
tag = H(KAB || m)
```

```
A invia: (m, tag)
```

```
Lato B (ricevente):
```

```
Ricalcola: tag' = H(KAB || m)
```

```
Confronta tag' con tag ricevuto
```

```
→ Se coincide: il messaggio è autentico e non è stato modificato
```

iii) Funzione di cifratura basata su hash:

Non è possibile usare H direttamente per cifrare (non è invertibile). Si usa H per costruire un keystream (simile a OFB) e poi XOR:

```
Cifratura (A):
```

```
Genera IV casuale
```

```
Per ogni blocco Mi:
```

```
Ci = Mi XOR H(KAB || IV || i) (i = indice del blocco)
```

```
Invia: (IV, C1 || C2 || ... || Cn)
```

```
Decifratura (B):
```

```
Per ogni blocco Ci:
```

```
Mi = Ci XOR H(KAB || IV || i)
```

7. Riepilogo Rapido

Concetto	In una riga
Hash function	Input arbitrario → output fisso, one-way
Preimage resistance	Da h non ricavo m
Second preimage resistance	Da m1 non trovo m2 con stesso hash
Collision resistance	Non trovo nessuna coppia m1≠m2 con stesso hash
MAC	Tag di autenticazione con chiave simmetrica condivisa
HMAC	MAC costruito con hash: H((K⊕opad)||H((K⊕ipad)||m))
Passi C per HMAC su N blocchi	N + 3
Differenza MAC vs Firma	MAC: chiave simmetrica, no non-ripudio. Firma: chiave privata, non-ripudio

Fonti: Slide Veltri (135-163) + Stallings Cap.11 (pp.637-710) e Cap.12 sez.12.1-12.5 (pp.711-742)